

Frequency-Augmented Multi-Scale Graph Neural Networks for Time-Series Industrial Control Network Attack Detection

YuLong Huang, Desi Wen, Tong Zhao *

School of Computer Science and Engineering, Northeastern University, Shenyang, 110819, China

* Corresponding author: 1535843978@qq.com

ABSTRACT

Industrial Control Systems (ICS) and Industrial Internet of Things (IIoT) networks are fundamentally different from traditional IT infrastructures. They include specialized components such as Programmable Logic Controllers, sensors and actuators that generate multivariate time-series data, making multivariate time-series anomaly detection a critical intrusion detection approach for operational technology environments. However, current methods have major limitations: they ignore the intrinsic multi-periodicity of industrial time series, anomaly-related frequency variations and cross-channel correlations. To overcome these drawbacks, we propose the Frequency-Augmented Multi-Scale Graph Neural Network. This framework adopts frequency decomposition, using adaptive graph convolution to process low-frequency components and capture multi-scale inter-series dependencies, while a time-channel module learns high-frequency temporal patterns. Extensive experiments on 2 real-world datasets verify that our model achieves state-of-the-art performance in precision, recall and F1-score, with stronger attack detection ability and remarkably lower computational cost.

Keywords: Industrial Control Systems (ICS), Industrial Internet of Things (IIoT), Anomaly detection, Multivariate time series, Graph Neural Networks (GNNs)

1. INTRODUCTION

Cyber-Physical Systems (CPS) are an integral part of modern society, with Industrial Control Systems (ICS) and the Industrial Internet of Things (IIoT) as their vital core components¹. They integrate physical processes with computational algorithms, networked sensors and actuators, boosting industrial efficiency and automation but also bringing unprecedented cybersecurity challenges². Distinct from traditional IT networks, ICS and IIoT devices generate time series data with strong periodicity and high predictability due to stable operational logic³. Leveraging these traits, time series anomaly detection can identify malicious activities in CPS in a protocol-agnostic manner, enabling seamless adaptation to various industrial scenarios.

Therefore, inspired by TimesNet⁴ and FEDformer⁵, we propose a graph neural network based on Fourier decomposition. This algorithm fully considers the multi-periodicity and anomaly-triggered high-frequency components in CPS scenarios. We decompose raw time series into periodic components dominated by low-frequency elements and trend components governed by high-frequency features using seasonal decomposition. Subsequently, Fast Fourier Transform (FFT) coupled with graph convolutions captures and represents diverse inter-series correlations evolving across varying temporal scales. Furthermore, for the non-dominant residual sequence, we employ a time-channel module to learn its temporal patterns and dimensional patterns. Finally, hybrid representations combining low and high frequency components predict input sequences for anomaly detection.

To summarize, the main contributions of our work are:

1. To model the multi-periodic characteristics inherent to industrial multivariate time series, our algorithm employs frequency-enhanced graph convolutions for multi-scale dependency extraction.
2. For the non-dominant residual sequence, a time-channel module is adopted to capture its temporal and dimensional patterns.

3. Extensive experiments demonstrate the proposed model's superiority over state-of-the-art baselines across diverse real-world datasets. Significantly, it detects substantially more network attacks from sensor time-series data than existing approaches.

The remainder of this article is structured as follows. In Section II, we provide a review of related work on time series anomaly detection. Our proposed anomaly detection architecture is introduced in Section III. Section IV presents the relevant experimental results and compares the influence of different architectures. In Section V, we summarize this article.

2. RELATED WORK

Time-series anomaly detection primarily employs prediction-based or reconstruction-based methods. Early prediction approaches leveraged RNN and Transformer⁶ architectures for sequential modeling. For parameter efficiency, convolutional networks emerged (e.g., TimesNet, PatchTST⁷), capturing local temporal features. With the emergence of deep learning-based time series analysis methods integrated with frequency domain techniques, FED trains an adversarial forecasting model using auxiliary information derived from the frequency domain for anomaly detection. While reconstruction-based methodologies demonstrate broader applicability than prediction-based approaches in industrial anomaly detection, significant technical evolution has occurred across several paradigms. Many approaches addressed data sparsity through variational frameworks like LSTM-VAE⁸. Concurrently, generative adversarial networks emerged as dominant solutions for handling data imbalance, with MAD-GAN⁹ and USAD¹⁰ leveraging adversarial learning for robust reconstruction. More recent innovations include contrastive learning architectures such as DCdetector¹¹, which constructs discriminative embedding spaces isolating anomalies. This progression highlights continual refinement in addressing core challenges of data scarcity, feature representation, and multivariate dependency modeling.

3. PROPOSED FRAMEWORK

3.1 Problem definition and Data Preprocessing

This paper addresses multivariate time-series anomaly detection as an unsupervised learning task. The model is trained exclusively on anomaly-free data to predict the future values of input series while minimizing prediction error. For normal inputs, predictions maintain high fidelity; however, anomalous inputs induce significant predictions deviations due to distributional shift.

Formally, let $X \in R^{N \times C}$ represent the multivariate time series with N timesteps and C channels (sensors). Using sliding windows of length w , we define windowed series at time t as $X_t = \{x_{t-w+1}, \dots, x_t\} \in R^{w \times C}$. The model $f: R^{w \times C} \rightarrow R^{w \times C}$ generates predictions:

$$\hat{X}_t = \{\hat{x}_t, \dots, \hat{x}_{t+w}\} \quad (1)$$

$$\hat{X}_t = f(X_t; \Omega). \quad (2)$$

Where Ω is the set of learnable parameters of the model.

For another unknown test time series $X_t^{test} = \{x_{t-w+1}^{test}, \dots, x_t^{test}\}$, we want to predict y_t . Here, $y_t \in \{0, 1\}$ where 1 denotes an anomalous data point and 0 denotes a normal data point.

To enhance model performance, we implement a comprehensive preprocessing pipeline featuring robust data normalization applied identically during training and testing. Reversible Instance Normalization (RevIN) is a normalization method specifically designed for time series forecasting. It addresses the distribution shift problem caused by temporal variations in mean and variance within time series data.

3.2 Input Embedding and Structure Overview

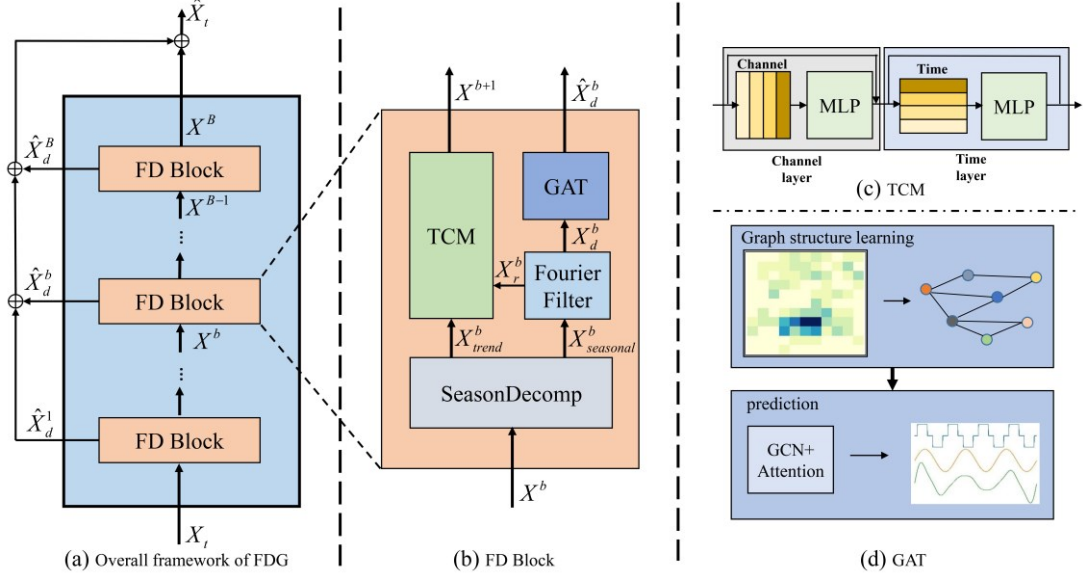


Figure 1. Architecture of the proposed model.

As illustrated in Fig. 1, the model architecture comprises a series of identical processing units termed FD (Fourier Decomposition) blocks. Each FD block integrates 4 core components: (1) a seasonal decomposition module, (2) a Fourier decomposition module, (3) a graph convolution module and (4) a time-channel module (TCM).

Unlike conventional GNN algorithms, positional encoding (PE) is incorporated into the raw time-series inputs to impart temporal awareness within the graph convolution module.

$$X_t = X_t + PE \quad (3)$$

Following prevalent architectural paradigms, we incorporate connections to interconnect the FD blocks. The entire process is mathematically formulated as follows:

$$X^b = FD(X^{b-1}). \quad (4)$$

where X^b denotes the output from the b th FD block, which serves as input to the $(b+1)$ th FD block. For the initial FD block, the input is designated as $X^0 = X_t$.

The seasonal component, trend component, dominant frequency component, and residual component in the FD block have strong physical meanings. Here, we introduce their roles and physical meanings in combination with the FD block shown in Fig. 1(b). When X^b is input into the b -th FD block, it is decomposed into the seasonal component $X_{seasonal}^b$ and the trend component X_{trend}^b by the seasonal decomposition module. Among them, the seasonal component $X_{seasonal}^b$ contains most of the periodic components in X^b , corresponding to the periodic operation and rhythmic changes in the industrial control system, and can be fed into the Fourier decomposition module to further extract its low-frequency periodic components. The trend component X_{trend}^b , together with the residual component X_r^b generated by the subsequent Fourier decomposition, reflects the drift characteristics of the industrial system and has high-frequency features. After passing through the Fourier decomposition module, X_d^b , which has stronger periodicity and is the dominant component of the original series X^b , is sent to the graph structure learning module to discover the topological relationships among various sensors. In contrast, the residual components X_{trend}^b and X_r^b with high-frequency

characteristics and a certain degree of nonlinearity is fed into the TCM module, where deep network methods are used to learn its nonlinear temporal and dimensional features. Each FD block finally outputs the prediction of the dominant component ($\hat{X}_d^b$) and the prediction of the residual component (X^{b+1}), where X^{b+1} is used as the input of the $(b+1)$ -th FD block to further extract the periodic components therein. Through this progressive process, the graph convolution learning of low-frequency periodic components and the deep nonlinear learning of high-frequency residual components are finally realized.

The prediction time series $\hat{X}_t$ can be formulated as the summation of all layers' graph convolution module predictions $\hat{X}_d^b$, aggregated nonlinear noise residues X^B .

$$\hat{X}_t = X^B + \sum_{b=1}^B \hat{X}_d^b. \quad (5)$$

B denotes the number of FD blocks.

3.3 Seasonal Decomposition

To extract the periodic and trend components from the input time series while facilitating multi-scale temporal analysis via FFT, we first decompose the input series using seasonal decomposition. This yields isolated periodic components that enhance FFT efficacy. Subsequent scale discovery is performed exclusively on the periodic components, as FFT demonstrates superior performance on cyclical data compared to raw inputs. The process is detailed as follows:

$$X_{trend}^b = AvgPool(padding(X^{b-1})). \quad (6)$$

$$X_{seasonal}^b = X^{b-1} - X_{trend}^b. \quad (7)$$

Among them, *avgpool* denotes the average pooling function, and *padding* denotes the padding function.

3.4 Fourier Decomposition

Fourier Decomposition detailed computational workflow is formalized as follows.

We compute the spectrum set $S = \{0, 1, \dots, w/2\}$ and its amplitude set F of $X_{seasonal}^b$ using the FFT.

$$F = Amp(FFT(X_{seasonal}^b)). \quad (8)$$

$$f_1, \dots, f_{k_f} = argTop(S), f_1, \dots, f_{k_f} \in F. \quad (9)$$

where, *FFT* and *Amp* denote the FFT and the calculation of amplitude values, respectively. k_f denotes the number of selected frequencies. To enhance learning of the periodic component, we apply the following filtering operation to $X_{seasonal}^b$.

$$X_d^b = FFT^{-1}(f_1, \dots, f_{k_f}). \quad (10)$$

$$X_r^b = X_{seasonal}^b - X_d^b. \quad (11)$$

where X_d^b represents the dominant component in time series $X_{seasonal}^b$.

3.5 Graph Structure Learning and Graph Convolution

Here, we employ the commonly used top- K method in Graph Convolution to learn the correlation between sensor nodes.

The i th sensor node v_i is derived via graph-based representation learning:

$$v_i \in R^e, i \in \{1, 2, \dots, C\}. \quad (12)$$

where e is the embedded dimensions of v_i . The embedding vectors v_i are initialized with random values and subsequently fine-tuned alongside the remainder of the model. These adjacency matrices are computed via the Top-K sparsification method as follows:

$$a_{ji} = \frac{v_i^T v_j}{\|v_i\| \bullet \|v_j\|}, \text{ for } j \in \{1, 2, \dots, C\} \setminus \{i\}. \quad (13)$$

$$A_{ji} = 1\{j \in \text{Top}K_{k_g}\}. \quad (14)$$

A_{ji} denotes the adjacency matrix, representing the connection weight from node j to node i . k_g represents the number of selected nodes.

$$H = \sigma(AX_d^b VW). \quad (15)$$

where H denotes the graph-convolved hidden features, W is a trainable weight matrix. V is all node embeddings.

We employ a weighted aggregation scheme to integrate the graph-convolved features H , where each H is weighted by the spectral amplitude at its corresponding frequency f_k :

$$a_1, \dots, a_{k_f} = \text{SoftMax}(f_1, \dots, f_{k_f}). \quad (16)$$

$$\hat{X}_d^b = \sum_{k=1}^{k_f} a_k H. \quad (17)$$

3.6 Time-Channel module

The time-channel module is designed to learn the temporal and channel patterns in the residual time series X_{trend}^b and X_r^b using two MLPs, with the specific calculation formulas given as follows:

$$X_{channel}^b = \text{MLP}_{channel}(X_{trend}^b + X_r^b)^T + (X_{trend}^b + X_r^b)^T. \quad (18)$$

$$X^b = \text{MLP}_{time}(X_{channel}^b) + X_{channel}^b. \quad (19)$$

3.7 Loss Function and Anomaly Detection

Our composite loss L comprises prediction losses from both time and frequency domains, specifically formulated as a weighted sum of two objective functions:

$$L = \frac{1}{w}(\alpha_1 \|X_{t+w} - \hat{X}_t\|_2^2 + \alpha_2 \|FFT(X_{t+w} - \hat{X}_t)\|_2). \quad (20)$$

Where α_1 and α_2 denote weighting coefficients. X_{t+w} is the true ground prediction of X_t .

To accelerate the detection pipeline during the inspection phase, we adopt the Mean Squared Error (MSE) as the anomaly quantification metric. For each test instance X_t^{test} in the testing set, the data point is flagged as an anomaly if its anomaly score s_t exceeds a predetermined threshold β .

$$y_i = \begin{cases} 1, & \text{if } s_i \geq \beta \\ 0, & \text{otherwise} \end{cases}. \quad (21)$$

4. EXPERIMENTS

In this section, we conduct experiments to answer the following research questions:

1. Does our method outperform baseline methods in accuracy of anomaly detection in multivariate time series, based on ground truth labelled anomalies?
2. Can our approach detect all types of network attacks?
3. How do the various components of the method contribute to its performance?
4. Can we discover relevant sensor through the learned graph structure?

4.1 Datasets and Metrics

We evaluated the algorithm on two real-world industrial scenario datasets: SWaT and WADI. The anomalies in SWaT and WADI are induced by cyber attacks.

We employ precision (Prec), recall (Rec), and F1-Score (F1) metrics over the test dataset and its corresponding ground truth values to assess the performance of our method and baseline models.

Notably, these metrics are calculated using the point adjustment strategy, in line with the approach used in most previous studies. The point adjustment strategy is well-suited for anomaly detection in industrial control scenarios, since anomalous events typically persist over multiple consecutive timesteps.

4.2 Experimental Setup

We implemented our method and baselines using PyTorch version 2.0.0, with CUDA 11.7 and the PyTorch Geometric Library version 1.5.0. The training was conducted on a Ubuntu server with Intel® Xeon® CPU E5-2640 @ 2.50GHz and an NVIDIA 4090 GPU. The models were trained utilizing the Adam optimizer with a learning rate of 0.0001. The entire network was trained with a batch size of 64 and for a total of 10 epochs. There are three FD blocks in total. The embedding dimension d was set to 64 across all block of model. We empirically set the sliding window size to 64 and the step to 1. The k_f is set to 2 and k_g is set to 10.

Table 1. Model performance in the testing set (%).

Algorithm	SWaT			WADI		
	P	R	F1	P	R	F1
MAD-GAN ⁹	97.89	78.98	87.42	92.10	69.40	79.15
OmniAnomaly ⁶	99.96	85.49	92.16	81.15	67.01	74.14
MTAD-GAT ¹²	99.99	65.66	79.27	88.11	59.31	70.89
GDN ¹²	99.35	85.12	91.68	89.30	54.70	67.84
TranAD ⁶	1	65.14	79.26	92.62	57.40	70.87
Anomaly Transformer ⁶	98.26	85.90	88.61	77.93	63.61	73.17
USAD ¹⁰	99.99	66.70	80.02	92.48	63.97	75.63
DCdetector ¹¹	98.83	85.34	91.59	95.94	78.53	86.36
ImDiffusion ¹³	89.88	84.665	87.09	90.58	83.43	88.87
CATCH ¹⁴	83.85	81.29	82.25	87.88	77.04	82.02
Ours	98.64	84.60	96.58	98.08	84.01	90.50

4.3 Accuracy

In Table 1, we show the anomaly detection metrics in terms of precision, recall and F1-score of our method and the baselines, on the 2 datasets. As evidenced by the results across 2 datasets, the proposed algorithm achieves state-of-the-art performance in the widely adopted F1-score metric. Furthermore, it attains the highest or second-highest precision and recall values on most datasets. This demonstrates our algorithm robust control over false positive rates and true positive rates across multiple pre-selected thresholds—a critical attribute for practical deployments, particularly in industrial control scenarios.

Comparative experiments reveal that in industrial control time series, cyclical constituents emerge as statistically predominant features, while residual trend components remain non-negligible; moreover, the frequency domain provides a more discriminative manifestation of anomalous occurrences.

4.4 Comprehensiveness

Our experimental evaluation benchmarks the proposed algorithm against state-of-the-art attack detection techniques using the SWaT and WADI industrial control datasets. Detection efficacy is quantified by the number of successfully identified attacks, with SWaT containing 36 distinct attack instances and WADI comprising 14 attacks (note: attacks 3 and 4 in WADI are jointly categorized as a single attack type).

As tabulated in Table 2, our algorithm achieves superior cyber-attack detection performance compared to baseline methods, particularly demonstrating enhanced capability in capturing temporal propagation patterns within time-series data.

Table 2. Comparison of attack detection techniques.

Algorithm	SWaT	WADI
MAD-GAN	16	8
OmniAnomaly	20	6
MTAD-GAT	20	6
GDN	22	7
TranAD	28	8
8Anomaly Transformer	22	6
USAD	24	6
DCdetector	23	8
ImDiffusion	24	8
CATCH	22	7
Ours	32	11

4.5 Ablation

To rigorously validate the necessity of each component in our methodology, we conduct progressive ablation studies observing performance degradation through systematic module exclusion on SWaT and WADI. We focus on the roles of frequency-domain decomposition, graph structure learning, and the TCM module.

First, we validate the effectiveness of the frequency-domain decomposition module. As shown in Table 3, the detection performance degrades more significantly when removing the seasonal decomposition than when removing the Fourier decomposition. These results demonstrate that Fourier decomposition requires strongly periodic linear series as input, which is exactly the preliminary effect achieved by the seasonal decomposition module. In this way, the Fourier decomposition module acts as a secondary filter for the seasonal component, further enhancing the linearity of the series fed into the graph convolution module. Next, we replace the graph structure learned by the graph structure learning module with a complete graph to verify its effectiveness. As can be observed from Table 3, the graph structure learning module contributes the most and serves as the core of the FD block. Finally, we remove the TCM module. It can be seen

that the TCM module contributes the least, indicating that periodic components dominate the time series generated by industrial control systems.

Table 3. Anomaly detection accuracy in term of precision(%), recall(%), and F1-score of ours and its variants

Method	SWaT			WADI		
	P	R	F1	P	R	F1
w/o Seasonal	96.82	75.37	84.75	81.27	78.63	79.93
w/o Fourier decomposition	97.92	77.51	86.52	85.74	75.73	80.42
complete graph	90.61	74.60	81.82	82.57	62.70	73.09
w/o TCM	97.66	83.20	89.85	95.89	81.36	88.02
Ours	98.64	84.60	96.58	98.08	84.01	90.50

4.6 Visualization of Graph Learning

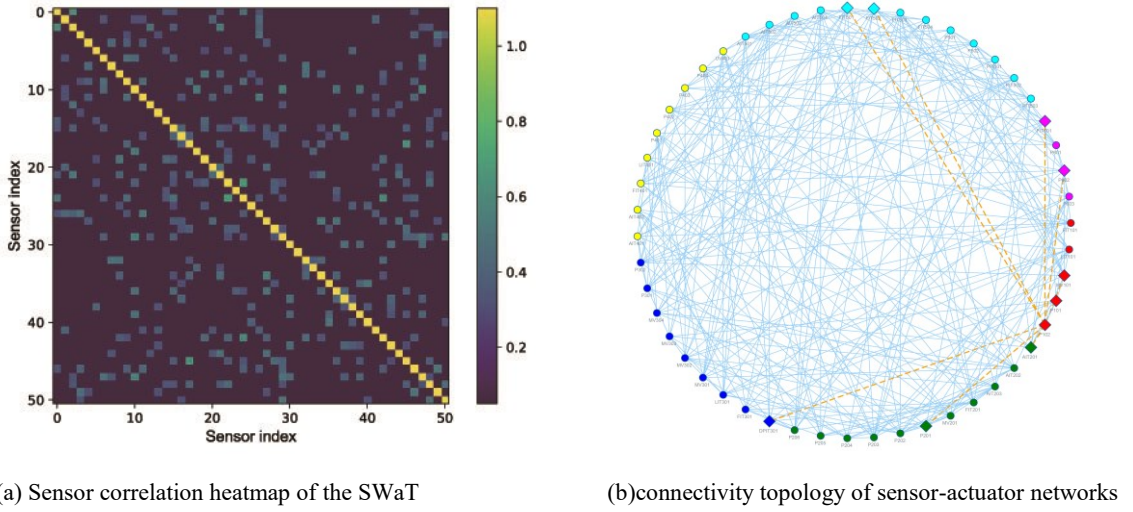


Figure 2. Visualization of the nodes.

Given the high dimensionality of sensors in the SWaT dataset, we compute aggregated sensor relationship heatmaps through consensus normalization of cosine distances a_{ji} adjacency matrices A . Taking the SWaT dataset as an example, In Fig. 2(a), the diagonal is the brightest, as each series is most strongly correlated with itself. Apart from the diagonal highlights, each row also exhibits adjacent or near-adjacent highlights, which is expected.

Understanding the sensor topology is helpful for identifying network attack targets. We can visualize the sensor topology. As shown in Fig. 2(b), we take the P102 sensor in SWaT as an example. In Fig. 2(b), we divide the nodes into 6 groups based on color, corresponding to the 6 stages of water treatment. The P102 sensor and its correlated sensors are highlighted with a diamond-shaped marker. The sensors correlated with P102 are MV101, P101, AIT201, FIT601, FIT502, P602, DPIT301, P201 and FIT501.

5. CONCLUSION

In this paper, we notice a deficiency in graph neural networks' ability to learn from ICS time-series directly, particularly in capturing the inter-sensor correlations crucial for effective anomaly detection. Therefore, we propose a graph neural network based on Fourier Decomposition. We decompose the original time-series using Seasonal decomposition and Fourier decomposition into the trend, dominant periodic components and residues, which are separately learned using MLPs and graph neural networks. Anomaly detection is performed using a prediction method. Through experimentation, our algorithm achieves superior detection performance on the SWaT and WADI datasets compared to other models with

satisfactory detection capabilities. Furthermore, compared to baseline algorithms, our algorithm can detect more network attacks, thus achieving enhanced network attack detection capabilities.

REFERENCES

- [1] Victor R. Kebande and Ali Ismail Awad. (2024) Industrial Internet of Things Ecosystems Security and Digital Forensics: Achievements, Open Challenges, and Future Directions. *ACM Comput. Surv.*, 56: 5-42. <https://doi.org/10.1145/3635030>.
- [2] Varadharajan, V., Tupakula, U., and Karmakar, K. K. (2024) Techniques for enhancing security in industrial control systems. *ACM Transactions on Cyber-Physical Systems*, 8(1): 1-36.
- [3] G. Cao, Y. Wu, D. Yu and Z. Wang.(2025) Attack Detection and Location Using State Forecasting in Multivariate Time Series of ICS. *IEEE Transactions on Network Science and Engineering*, 12: 2989-3001, doi: 10.1109/TNSE.2025.3555764.
- [4] Malhotra, P., TV, V., Vig, L., Agarwal, P., and Shroff, G. (2017). TimeNet: Pre-trained deep recurrent neural network for time series classification. *arXiv preprint arXiv:1706.08838*.
- [5] Zhou T, Ma Z, and Wen Q. (2022) Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*. PMLR, 27268-27286.
- [6] Ahmed, S., Nielsen, I. E., Tripathi, A., Siddiqui, S., Ramachandran, R. P., and Rasool, G. (2023). Transformers in time-series analysis: A tutorial. *Circuits, systems, and signal processing*, 42(12): 7433-7466.
- [7] Feizi, H., and Sattari, M. T. (2026). Streamflow Forecasting Based on PatchTST, LSTM, and Ensemble Learning Approaches. *Water Resources Management*, 40(1): 44.
- [8] Ma, C., Wang, Y., Li, F., Zhang, H., Zhang, Y., and Zhang, H. (2024). Constructing Attention-LSTM-VAE Power Load Model Based on Multiple Features. *Advances in Mathematical Physics*, 2024(1), 1041791.
- [9] Fatima, S. S. W. (2024). MAD-GAN: Multi-Agent Decomposition with GANs for Time Series Forecasting in Manufacturing Systems (Master's thesis, University of Windsor (Canada)).
- [10] Audibert, J., Michiardi, P., Guyard, F., Marti, S., and Zuluaga, M. A. (2020, August). Usad: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining* pp. 3395-3404.
- [11] Zhou, W. (2022). DCDetector: An IoT terminal vulnerability mining system based on distributed deep ensemble learning under source code representation. *arXiv preprint arXiv:2211.16235*.
- [12] Chen, H., and Eldardiry, H. (2024). Graph time-series modeling in deep learning: A survey. *ACM Transactions on Knowledge Discovery from Data*, 18(5): 1-35.
- [13] Chen, Y., Zhang, C., Ma, M., Liu, Y., Ding, R., Li, B., and Zhang, D. (2023). Imdiffusion: Imputed diffusion models for multivariate time series anomaly detection. *arXiv preprint arXiv:2307.00754*.
- [14] Wu, X., Qiu, X., Li, Z., Wang, Y., Hu, J., Guo, C., and Yang, B. (2024). Catch: Channel-aware multivariate time series anomaly detection via frequency patching. *arXiv preprint arXiv:2410.12261*.